LEAF: A Learning-Enabled ADMM Framework for Accelerated Convex Optimization

Anonymous Authors

***Abstract*—We propose LEAF, a learning-enabled ADMM framework for accelerated convex optimization. The key idea is to approximate the Moreau envelope of the objective function using an Input Convex Neural Network (ICNN), resulting in a learned model that preserves convexity and smoothness. This leads to the proposed Moreau Envelope Learning ADMM (MEL-ADMM) and its splitting variant sMEL-ADMM. Unlike existing approaches that learn high-dimensional operators directly, LEAF learns a scalar-valued Moreau envelope, significantly reducing model complexity and improving data efficiency. The framework accommodates a broad class of convex problems with smooth and non-smooth objectives. By embedding convexity explicitly through the ICNN architecture, the proposed approach maintains high approximation accuracy while preserving key structural properties of the optimization problem. Both MEL-ADMM and sMEL-ADMM are developed with theoretical guarantees of convergence and feasibility under the learned model. Rigorous analysis shows that the proposed methods achieve convergence rates comparable to classical ADMM while reducing per-iteration computational cost. Numerical experiments demonstrate up to an order-of-magnitude speedup over state-of-the-art solvers while maintaining low optimality gaps.**

***Index Terms*—Learning-to-optimize; Input convex neural networks; Moreau envelope; ADMM; Convex optimization**

I. Introduction

Learning-to-optimize (L2O) is an emerging paradigm that integrates machine learning with numerical optimization to improve the efficiency of solving complex problems. [1], [2] Instead of relying solely on iterative algorithms, L2O leverages data to learn mappings from problem instances to solutions or to create fast numerical iterations based on existing solvers. In particular, for parametric optimization problems where the objective and constraints depend on varying parameters, L2O methods can exploit shared structures across instances to provide near-optimal solutions with reduced computational cost. By learning from previously solved examples, these approaches enable scalable and adaptive optimization frameworks that are especially well-suited for real-time and large-scale applications. The main challenges of L2O methods lie in the accuracy of the obtained solutions, lack of constraint satisfaction guarantees, and their dependence on the training data distribution for convergence. As a result, their performance often degrade on out-of-distribution problem instances, leading to unreliable or inaccurate solutions. Moreover, the lack of theoretical guarantees on feasibility and convergence limits their robustness in safety-critical applications. Integrating more problem and algorithmic structures into L2O methods could potentially overcome these challenges.

Over the past decades, the Alternating Direction Method of Multipliers (ADMM) has attracted significant research attention and has undergone substantial theoretical and algorithmic developments [3], [4], [5]. As a dual domain method, the ADMM is superior to its primal domain counterparts, such as the gradient descent method, in terms of convergence speed. At a high level, ADMM is useful when an objective consists of multiple components that depend on different variables, while those variables must satisfy local and possibly coupling constraints. Due to its broad capability, ADMM has been ubiquitous in many applications, including image processing [6], power grid systems [7], communication networks [8], neural network training [9], and cooperative planning of multiple robots [10]. Despite these advantages, ADMM faces notable limitations in large-scale parametric optimization, where problem instances vary due to changing initial conditions or parameters. In such settings, each instance is typically solved independently, without exploiting any shared structure, resulting in substantial computational overhead, especially for high-dimensional or non-smooth problems requiring many iterations.

In this paper, we propose LEAF (Learning-Enabled ADMM Framework), a *structured L2O framework* that leverages machine learning techniques based on ADMM to address the above challenges of both ADMM and L2O methods. The scalability of ADMM for parametric convex optimization is substantially improved by our L2O method, which is purposely designed to integrate the problem and ADMM structures to enhance solution accuracy, feasibility, and convergence guarantees. In particular, this work develops a supervised learning model based on Input Convex Neural Networks (ICNNs) to approximate the Moreau envelope (ME) of the convex objective function, yielding a scalar output and guaranteeing the smoothness and convexity of the ME. The model is then embedded in ADMM to accelerate its computation while preserving feasibility and convergence. LEAF supports a wide range of optimization problems, including non-smooth formulations with dynamic constraints. Compared with existing L2O methods, LEAF significantly reduces the size of the learning model by leveraging ICNNs, which preserve convexity and maintain high accuracy with fewer trainable parameters. This compact architecture not only lowers computational and memory requirements but also improves data efficiency by embedding convexity priors directly into the network structure, thereby reducing model complexity and enabling accurate learning from limited training samples. Based on LEAF, we propose two algorithms, Moreau-Envelope Learning ADMM (MEL-ADMM) and splitting MEL-ADMM (sMEL-ADMM), that ensure both global convergence and feasibility of the solution. Rigorous theoretical analysis shows that the convergence rate of the proposed algorithms is comparable to ADMM, while achieving a lower per-iteration computational

cost. In several applications, our methods consistently outperform existing methods, achieving up to ten times faster solving time within acceptable optimality gaps, even with relatively compact learning models.

Our main contributions are summarized as follows:

- *An effective learning-enabled ADMM framework (LEAF)*: We propose a general framework that integrates supervised learning into ADMM to bypass expensive primal updates through efficient forward computations.
- *Convexity informed learning*: We employ ICNNs to approximate Moreau envelopes while explicitly enforcing convexity and smoothness, leading to highly accurate and data-efficient learning.
- *Theoretical guarantees:* We propose two algorithms with rigorous theoretical analysis ensuring both global convergence and feasibility of the solution.
- *Comprehensive experimental validation*: Numerical experiments on large-scale convex optimization problems demonstrate significant computational speedups over state-of-the-art solvers.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the problem formulation and preliminaries. Section IV introduces LEAF and the associated algorithms. Section V provides theoretical analysis and convergence guarantees. Section VI presents an MPC application of LEAF. Section VII reports numerical experiments. Finally, Section VIII concludes the paper.

II. Literature review

This section reviews the learning-to-optimize (L2O) literature most relevant to this work, focusing on three aspects: (i) learning approaches, (ii) modeling strategies and structural priors, and (iii) feasibility and optimality considerations.

Learning Approaches. The integration of machine learning into optimization to improve computational efficiency and scalability has been extensively studied [1], [11]. Existing approaches can be broadly categorized into three classes.

- The first class consists of end-to-end learning methods that directly map problem parameters to optimal solutions. These methods typically rely on supervised learning and require a large number of training samples generated by conventional solvers [12], [13]. To improve solution quality, several works incorporate optimality conditions, such as the Karush–Kuhn–Tucker (KKT) conditions, into the training process [14], [15]. Constraint satisfaction is often addressed through penalty-based losses or feasibility-seeking mechanisms [16], [17]. However, these methods generally lack strict guarantees of feasibility and optimality, and their performance is sensitive to the coverage of the training data distribution. As a result, they may exhibit degraded performance when encountering out-of-distribution problem instances.
- The second class enhances existing optimization solvers by incorporating learned components within the iterative process. Representative works include learning warm-start initializations [18], [19] and predicting active constraint sets to facilitate constraint elimination [20]. These methods share a common objective of reducing the number of iterations required for convergence while preserving the underlying solver structure. Another line of research focuses on approximating computationally expensive operators within each iteration using data-driven models [21], [22]. While effective in accelerating individual steps, these methods typically involve learning high-dimensional mappings whose outputs match the dimension of the decision variables, which can limit scalability and weaken structural guarantees. In contrast, our framework adopts a different strategy by learning the Moreau envelope rather than directly approximating the proximal operator. Our approach yields a learning model with a scalar-valued output, significantly reducing learning complexity while preserving key structural properties such as convexity.
- The third class designs end-to-end learning architectures by unrolling the iterations of classical optimization algorithms into trainable networks. Representative examples include ADMM-based deep learning models [23], [24] and learned optimizers based on gradient descent dynamics [25], [26]. While these methods can achieve efficient inference by learning optimization dynamics, they often lack rigorous convergence guarantees and typically require a substantial number of training samples.

End-to-end learning methods generally heavily depend on the coverage of the training dataset, which must include representative problem instances and their corresponding true optimal solutions as labels. In high-dimensional settings, such coverage is often limited, which can reduce the accuracy and generalization performance of end-to-end learning models.

Modeling approach and structural priors. The aforementioned studies rely on generic neural network models to directly infer solutions or approximate intermediate operators. These approaches typically produce outputs with the same dimensionality as the decision variables, leading to high-dimensional representations. As a result, they often suffer from substantial computational complexity, limited scalability, and data inefficiency. To address these limitations, we propose a structured learning approach that leverages supervised learning with input-convex neural networks (ICNNs) to approximate the Moreau envelope of the objective function. Unlike generic models, our model learns a scalar-valued function, significantly reducing the output dimensionality. Moreover, by embedding convexity through ICNNs, the learned model preserves key structural properties, including convexity, smoothness, and Lipschitz continuity, providing stronger theoretical guarantees and improved approximation accuracy. Overall, this design significantly improves computational efficiency, scalability, accuracy, and data efficiency.

Feasibility and optimality. While end-to-end L2O approaches can deliver fast solutions, they often do not guarantee satisfaction of problem constraints. Several constraint-enforcement approaches have been proposed to mitigate this issue, including penalty methods [16], [27] and projection-based methods [28]. The former adds penalty terms to the loss function to penalize constraint violations during training; however, it lacks feasibility guarantees. The latter projects the predicted

solutions onto the feasible set to provide feasible solutions at the expense of larger optimality gaps. Our methods guarantee solution feasibility while achieving optimality gaps below 0.1% in practical examples.

III. Problem Formulation

Before detailing the problem formulation, we introduce the notations and important definitions used in this paper. Let $\mathbb{N}_{>0}$ denote the set of positive integers. For a function f, $\mathrm{Dom}(f)$ denotes its domain. Given an index set $\mathcal{I} = \{i_1, i_2, \ldots, i_n\} \subset \mathbb{N}_{>0}$ and real matrices $M_{i_1}, M_{i_2}, \ldots, M_{i_n}$ in appropriate dimensions, let us denote their concatenation

$$[M_i]_{i\in\mathcal{I}} = \begin{bmatrix} M_{i_1}^\top & M_{i_2}^\top & \ldots & M_{i_n}^\top \end{bmatrix}^\top.$$

A function $f : X \to \mathbb{R} \cup \{\pm\infty\}$ is proper if $\mathrm{Dom}(f) \neq \emptyset$ and $f(x) > -\infty$ for all $x \in X$, is lower semi-continuous (l.s.c) at x if $x_k \to x$ implies $\lim_{k\to\infty} f(x_k) \geq f(x)$, and is l.s.c. on X if f is l.s.c. at every $x \in X$. A function f is convex if $\forall x, y \in \mathrm{Dom}(f)$ and $\alpha \in [0, 1]$, $\alpha x + (1-\alpha)y \in \mathrm{Dom}(f)$ and $f(\alpha x + (1-\alpha)y) \leq \alpha f(x) + (1-\alpha)f(y)$. Let $\Gamma_0(X)$ denote the set of all proper, lower semi-continuous, and convex functions on X [29]. The indicator function of a convex set $\mathcal{C}$ is defined as $I_{\mathcal{C}}(x) = +\infty$ if $x \notin \mathcal{C}$ and $I_{\mathcal{C}}(x) = 0$ if $x \in \mathcal{C}$. A function f is L-Lipschitz continuous with Lipschitz constant L if $\|f(x) - f(y)\|_2 \leq L\|x - y\|_2$ for all $x, y \in \mathrm{Dom}(f)$.

A. Optimization problem

This paper considers the following optimization problem

$$\text{minimize } f(z) \tag{1a}$$
$$\text{s.t. } Az = b, \tag{1b}$$
$$z \in \mathcal{Z}, \tag{1c}$$

where $z \in \mathbb{R}^n$ is the decision vector. We assume that the objective function $f(z) : \mathbb{R}^n \to \mathbb{R} \cup \{\pm\infty\}$ is convex, proper, and lower semi-continuous, that is, $f \in \Gamma_0(\mathbb{R}^n)$. The matrix $A \in \mathbb{R}^{m\times n}$, vector $b \in \mathbb{R}^m$, and non-empty convex set $\mathcal{Z}$ are problem parameters and may vary across problem instances. We assume that the parameters $(A, b, \mathcal{Z})$ are such that the optimization problem (1) is feasible.

By introducing auxiliary variable $w \in \mathbb{R}^n$, the problem (1) is equivalent to

$$\text{minimize } f(z) + I_{\bar{\mathcal{Z}}}(w) \tag{2a}$$
$$\text{s.t. } w - z = 0, \tag{2b}$$

where $\bar{\mathcal{Z}} = \{z \in \mathcal{Z} \mid Az = b\}$. The augmented Lagrangian of (2) with the penalty parameter $\rho > 0$ is given by

$$\mathcal{L}(z, w, \alpha) = f(z) + I_{\bar{\mathcal{Z}}}(w) + \alpha^\top(w - z) + \frac{\rho}{2}\|w - z\|_2^2. \tag{3}$$

Applying the over-relaxed ADMM iterations [30] to the augmented Lagrangian (3) yields

$$z^{i+1} = \arg\min f(z) + \frac{\rho}{2}\|w^i - z + \rho^{-1}\alpha^i\|_2^2, \tag{4a}$$
$$\tilde{z}^{i+1} = \gamma z^{i+1} + (1-\gamma)w^i, \tag{4b}$$
$$w^{i+1} = \Pi_{\bar{\mathcal{Z}}}(\tilde{z}^{i+1} - \rho^{-1}\alpha^i), \tag{4c}$$
$$\alpha^{i+1} = \alpha^i + \rho(w^{i+1} - \tilde{z}^{i+1}), \tag{4d}$$

where $\gamma \in (0, 2)$ is the relaxation parameter and $\Pi_{\bar{\mathcal{Z}}}$ denotes the Euclidean projection onto $\bar{\mathcal{Z}}$. We tune γ and ρ, specifically we choose $\gamma \in [1.0, 1.8]$, which has been empirically shown to improve the convergence rate [31], [32].

The computational bottleneck of the ADMM iteration (4) is primarily associated with the subproblems (4a) and (4c). This burden is exacerbated when f is non-quadratic or non-smooth, in which case solving (4a) becomes particularly expensive.

Addressing the above challenge, our **objective** is to efficiently compute *feasible and near-optimal solutions* to (1) by leveraging machine learning to accelerate ADMM iterations while preserving convergence and feasibility guarantees.

B. Moreau envelopes

We recall the concept of Moreau envelopes. Let $f : \mathbb{R}^n \to \mathbb{R} \cup \{+\infty\}$ be a closed proper convex function. The proximal operator $\mathbf{prox}_{\lambda f} : \mathbb{R}^n \to \mathbb{R}$ of the scaled function λf, for $\lambda > 0$, is given by

$$\mathbf{prox}_{\lambda f}(x) = \arg\min_z \left\{ f(z) + \frac{1}{2\lambda}\|x - z\|_2^2 \right\}. \tag{5}$$

The Moreau envelope $M_\lambda f$ of f is then defined as [33]

$$M_\lambda f(x) = \min_z \left\{ f(z) + \frac{1}{2\lambda}\|x - z\|_2^2 \right\}. \tag{6}$$

Proposition 1 ([33], [34]): If $f \in \Gamma_0(\mathbb{R}^n)$ then

(i) $M_\lambda f(x)$ is convex and $\nabla M_\lambda f(x)$ is Lipschitz continuous with constant $\frac{1}{\lambda}$,
(ii) $M_\lambda f(x) \leq f(x)$ for all $x \in \mathbb{R}^n$.

Furthermore, a useful relationship exists between $M_\lambda f(x)$ and $\mathbf{prox}_{\lambda f}(x)$, expressed by [34, Proposition 5.1.7]:

$$\mathbf{prox}_{\lambda f}(x) = x - \lambda \nabla M_\lambda f(x). \tag{7}$$

With the help of the Moreau envelope, substituting (7) into (4a) with $\lambda = \frac{1}{\rho}$, the $z-$update (4a) turns into

$$z^{i+1} = q^i - \frac{1}{\rho}\nabla M_{\frac{1}{\rho}} f(q^i), \tag{8}$$

where $q^i = w^i + \rho^{-1}\alpha^i$.

IV. Learning-Enabled ADMM Framework (LEAF)

To address the computational bottleneck in ADMM iterations, particularly the expensive proximal update in (4a), we propose a *learning-enabled ADMM framework* (LEAF) that replaces this step with a learned approximation. The key idea is to approximate the Moreau envelope (ME) of the objective function using a differentiable learning model, enabling the proximal update to be computed efficiently via gradient evaluation. Rather than learning to directly approximate the proximal operator by a neural network as in [22], our approach learns a scalar-valued representation of the ME $M_{\frac{1}{\rho}} f$, from which the required update can be obtained through its gradient by (8). This formulation significantly reduces the learning complexity while preserving important structural properties. To ensure convexity and theoretical consistency, we employ input convex neural networks (ICNNs) to model the ME. Building on this idea, we develop the Moreau Envelope

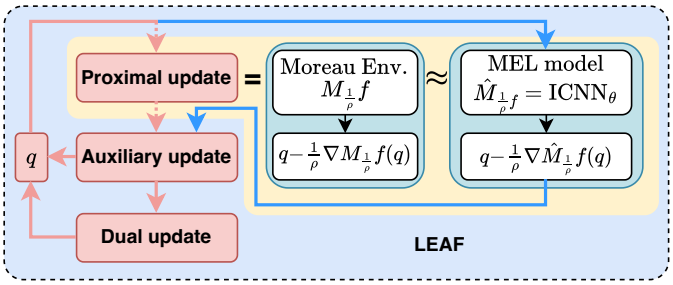


Fig. 1: Schematic overview of the Learning-Enabled ADMM Framework (LEAF).

Learning (MEL) approach and integrate it into ADMM to obtain two algorithms, termed MEL-ADMM and its splitting variant sMEL-ADMM, to be presented in this section.

A. Moreau envelope learning (MEL) model

To efficiently approximate the Moreau envelope $M_{\frac{1}{\rho}}f$ and its gradient $\nabla M_{\frac{1}{\rho}}f$, we develop a neural network-based model $\hat{M}_{\frac{1}{\rho}}f$ of $M_{\frac{1}{\rho}}f$ tailored for integration within ADMM iterations. Previous studies that use Gaussian process models [5], [35] suffer from increasing inference cost as the dataset grows and lack theoretical guarantees, making them unsuitable for fast iterative optimization. Neural networks enable fast inference and gradient evaluations, making them well-suited for replacing computationally expensive primal updates.

Since the ME is convex, the learning model should preserve this property to ensure consistency with the underlying optimization problem. To enforce convexity, we employ an ICNN [36], which guarantees that the learned function is convex with respect to its input. The structure of an ℓ-layer ICNN with input x and output $\mathrm{ICNN}_\theta(x)$ is given by

$$\hat{M}_{\frac{1}{\rho}}f(x): \begin{cases} v_0 = \phi_0(W_0 x + b_0), \\ v_{j+1} = \phi_j(W_j v_j + V_j x + b_j), \\ \qquad\qquad j = 0, \dots, \ell - 1, \\ \mathrm{ICNN}_\theta(x) = \phi_\ell(W_\ell v_\ell + V_\ell x + b_\ell), \end{cases} \tag{9}$$

where v_0 represents the activation of the initial hidden layer, W_0 is the weight matrix connecting the input x to this first layer, ϕ_j is the nonlinear activation function for layer j, $\theta = (\{W_j, b_j\}_{j=0,\dots,\ell}, \{V_j\}_{j=1,\dots,\ell})$ denotes the set of all parameters. All functions ϕ_j are convex and non-decreasing, and all W_j are non-negative. Under these conditions, $\mathrm{ICNN}_\theta(x)$ is a convex function of x. This architecture ensures convexity by constraining weights and activation functions, allowing the learned ME model $\hat{M}_{\frac{1}{\rho}}f$ to inherit key properties of the ME. Unlike direct proximal operator learning [22], this approach leverages ICNNs to learn the scalar-valued ME, reducing output dimensionality and improving scalability. The overall framework is illustrated in Fig. 1, where the learned ME model $\hat{M}_{\frac{1}{\rho}}f$ serves as an approximation of the ME $M_{\frac{1}{\rho}}f$, from which the proximal update is computed.

If the objective function f is strongly convex, its ME $M_{\frac{1}{\rho}}f$ also inherits strong convexity, which can be exploited to further improve the learning model.

Algorithm 1 The MEL-ADMM algorithm

Require: Parameters $\rho > 0$, $\gamma \in (0, 2)$, initial points w^0 and α^0, and trained MEL model $\hat{M}_{\frac{1}{\rho}}f$

1: **repeat**
2: $\quad \hat{z}^{i+1} = w^i + \rho^{-1}\alpha^i - \frac{1}{\rho}\nabla \hat{M}_{\frac{1}{\rho}}f(w^i + \rho^{-1}\alpha^i)$
3: $\quad \tilde{z}^{i+1} = \gamma \hat{z}^{i+1} + (1-\gamma)w^i$
4: $\quad w^{i+1} = \Pi_{\bar{\mathcal{Z}}}(\tilde{z}^{i+1} - \rho^{-1}\alpha^i)$
5: $\quad \alpha^{i+1} = \alpha^i + \rho(w^{i+1} - \tilde{z}^{i+1})$
6: **until** The termination criterion is satisfied

Definition 1 (Strong Convexity): A function $f \in \Gamma_0(\mathbb{R}^n)$ is strongly convex with modulus σ (σ-strongly convex) if and only if there exists a constant $\sigma > 0$ such that $f - \frac{\sigma}{2}\|\cdot\|^2$ is convex. That is for all $\lambda \in (0,1)$ and for all $x, y \in \mathbb{R}^n$,

$$f(\lambda x + (1-\lambda)y) \le \lambda f(x) + (1-\lambda)f(y) - \tfrac{\sigma}{2}\lambda(1-\lambda)\|x-y\|^2.$$

Lemma 1: [37, Lemma 2.23] The function f is strongly convex if and only if its Moreau envelope is strongly convex.

Therefore, when f is strongly convex, the above result suggests a stronger structural prior for learning the ME by combining the ICNN_θ with a quadratic function.

Finally, to ensure Lipschitz continuity of the gradient $\nabla M_{\frac{1}{\rho}}f$, we select activation functions that are non-decreasing, twice differentiable, and have bounded derivatives. This choice guarantees smoothness of the learned ME model, which is critical for stable ADMM updates.

In a nutshell, the proposed neural network based framework for learning the ME $M_{\frac{1}{\rho}}f$ is summarized in Table I.

Remark 1: If a function is σ-strongly convex, then it is also σ'-strongly convex for all $0 < \sigma' \le \sigma$. This allows us to determine the strong convexity modulus σ_M of the ME. In particular, the largest $\sigma_M > 0$ such that $M_{\frac{1}{\rho}}f - \frac{\sigma_M}{2}\|.\|_2^2$ is convex will be used.

B. Training the MEL model

To train the MEL model, we construct a dataset $\{(q^i, M_{\frac{1}{\rho}}f(q^i), \nabla M_{\frac{1}{\rho}}f(q^i))\}_{i=1,\dots,n_D}$ obtained from the ADMM iteration (4), where each sample consists of a query point $q_i = w^i + \rho^{-1}\alpha^i$, the corresponding ME value $M_{\frac{1}{\rho}}f(q^i)$, and its gradient $\nabla M_{\frac{1}{\rho}}f(q^i)$. The ME value and its gradient are determined by the minimum value of (4a) and by (7). To systematically collect the data across various initial conditions, we sample the initial points $w^0 \in \mathcal{Z}$ and $\alpha^0 \in \mathbb{R}^n$ using normal distributions, then execute the iteration (4) to obtain the data. The data collection is performed offline, allowing us to gather as much data as needed to train the MEL model.

The training loss function consists of the output loss term

$$\mathcal{L}_{\text{out}} = \sum_{i=1}^{n_D} \left(\hat{M}_{\frac{1}{\rho}}f(q^i) - M_{\frac{1}{\rho}}f(q^i)\right)^2$$

and a gradient loss term for matching the ME gradient values $\nabla M_{\frac{1}{\rho}}f(q^i)$ in the training data

$$\mathcal{L}_{\text{grad}} = \sum_{i=1}^{n_D} \left(\mu_g \left\|\nabla \hat{M}_{\frac{1}{\rho}}f(q^i) - \nabla M_{\frac{1}{\rho}}f(q^i)\right\|_2^2\right.$$

Function f	ME $M_{\frac{1}{\rho}}f$	Neural network model	Activation function
Convex	Convex, ρ-Lipschitz continuous gradient	ICNN_θ	non-decreasing, smooth,
Strongly convex	Strongly convex, ρ-Lipschitz continuous gradient	ICNN_θ+ $\frac{\sigma_M}{2}\|.\|_2^2$	ϕ_j' and ϕ_j'' are bounded.

TABLE I: Configurations for learning the Moreau envelope of a convex function.

$$+ \mu_p \| \max(0, \hat{M}_{\frac{1}{\rho}} f(q^i) - f(q^i))\|_2^2 \Big). \quad (10)$$

In (10), the weights μ_g and μ_p are positive, and the second term $\| \max(0, \cdot)\|_2^2)$ is designed to enforce property (ii) in Proposition 1 during the training process.

Putting everything together, the MEL model $\hat{M}_{\frac{1}{\rho}} f$ is trained by minimizing the total loss function

$$\underset{\theta}{\text{minimize}} \ \mathcal{L}_{\text{out}} + \mathcal{L}_{\text{grad}}$$

subject to W_i being element-wise nonnegative.

C. The MEL-ADMM algorithm

Once the MEL model $\hat{M}_{\frac{1}{\rho}} f$ is trained, it is integrated into the ADMM iteration (4) by replacing the z-update in (4a) with

$$\hat{z}^{i+1} = q^i - \rho^{-1} \nabla \hat{M}_{\frac{1}{\rho}} f(q^i). \quad (11)$$

Here we use $\hat{z}$ to distinguish the predicted decision vector z from the exact one. The resulting algorithm is termed MEL-ADMM and is presented in Algorithm 1.

Compared to the ADMM iteration (4), the MEL-ADMM has a significant computational advantage. Specifically, instead of solving the potentially expensive optimization problem (4a) of the z-update, the $\hat{z}$-update in (11) of MEL-ADMM is an inexpensive forward computation (inference) of the neural network model $\hat{M}_{\frac{1}{\rho}} f$, thus the computation time can be shortened. In addition, the MEL model is independent of the problem instance, therefore, once trained, MEL-ADMM can be applied to solve any instance of (1) without retraining the MEL model.

Observe that $\bar{\mathcal{Z}}$ incorporates both a convex set $\mathcal{Z}$ and an affine equality constraint, hence the projection onto $\bar{\mathcal{Z}}$ in (4c) at each iteration can be computationally expensive. The MEL-ADMM algorithm can be further accelerated by treating these constraints separately, as presented next.

D. The splitting MEL-ADMM (sMEL-ADMM) algorithm

The constraints (1b) and (1c) can be split into two sets by introducing another auxiliary variable $v \in \mathbb{R}^n$, leading to an equivalent form of problem (1) as

$$\text{minimize } f(z) + I_{\mathcal{Z}}(w) + I_{\mathcal{A}}(v) \quad (12a)$$
$$\text{s.t. } w - z = 0, \quad (12b)$$
$$w - v = 0, \quad (12c)$$

where $\mathcal{A} = \{z \in \mathbb{R}^n | Az = b\}$. The augmented Lagrangian of (12) is expressed as

$$L(z, w, v, \alpha, \beta) = f(z) + I_{\mathcal{Z}}(w) + I_{\mathcal{A}}(v) + \alpha^\top (w - z) + \frac{\rho}{2}\|w - z\|_2^2 + \beta^\top (w - v) + \frac{\rho_v}{2}\|w - v\|_2^2.$$

Algorithm 2 The sMEL-ADMM algorithm

Require: Parameters $\rho > 0$, $\gamma \in (0, 2)$, initial points v^0, α^0, β^0, and trained MEL model $\hat{M}_{\frac{1}{\rho}} f$.

1: **repeat**
2: $\quad \hat{z}^{i+1} = w^i + \rho^{-1}\alpha^i - \frac{1}{\rho}\nabla \hat{M}_{\frac{1}{\rho}} f(w^i + \rho^{-1}\alpha^i)$
3: $\quad \tilde{z}^{i+1} = \gamma \hat{z}^{i+1} + (1 - \gamma) w^i$
4: $\quad w^{i+1}, v^{i+1}$, α^{i+1}, and β^{i+1} by (13c)-(13f), (14)
5: **until** Termination criterion is satisfied

Inspired by operator splitting methods [38] and based on the reformulation (12), we propose a method named splitting MEL-ADMM (sMEL-ADMM) in the following form

$$\begin{aligned} z^{i+1} &= \arg\min f(z) + \frac{\rho}{2}\|w^i - z + \rho^{-1}\alpha^i\|_2^2 \\ &= \mathbf{prox}_{\frac{1}{\rho} f}\left(w^i + \rho^{-1}\alpha^i\right) && (13a) \\ \tilde{z}^{i+1} &= \gamma z^{i+1} + (1 - \gamma) w^i, && (13b) \\ w^{i+1} &= \underset{w \in \mathcal{Z}}{\arg\min} \|w - \tilde{z}^{i+1} + \rho^{-1}\alpha^i\|_2^2 + \|w - v^i + \rho_v^{-1}\beta^i\|_2^2 \\ &= \Pi_{\mathcal{Z}}\left(\frac{\tilde{z}^{i+1} + v^i - \rho^{-1}\alpha^i - \rho_v^{-1}\beta^i}{2}\right), && (13c) \\ v^{i+1} &= \Pi_{\mathcal{A}}\left(w^{i+1} + \rho^{-1}\beta^i\right), && (13d) \\ \alpha^{i+1} &= \alpha^i + \rho(w^{i+1} - \tilde{z}^{i+1}), && (13e) \\ \beta^{i+1} &= \beta^i + \rho_v(w^{i+1} - v^{i+1}). && (13f) \end{aligned}$$

Note that the z-update (13a) can be replaced by the MEL model as in MEL-ADMM. Since $\mathcal{A}$ is an affine set, the v-update (13d) admits a closed-form solution by directly solving the KKT conditions as follows:

$$\begin{bmatrix} I & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} v^{i+1} \\ \lambda^{i+1} \end{bmatrix} = \begin{bmatrix} w^{i+1} + \rho^{-1}\beta^i \\ b \end{bmatrix}, \quad (14)$$

where $\lambda^{i+1} \in \mathbb{R}^m$ is the Lagrange multiplier associated with the equality constraint $Av = b$. The matrix on the left-hand side can be decomposed, e.g., using LU decomposition, prior to the ADMM iterations, allowing the linear system (14) to be solved efficiently at each iteration.

Employing the MEL model $\hat{M}_{\frac{1}{\rho}} f$ in (13a) and the closed-form v-update in (14), the sMEL-ADMM algorithm is presented in Algorithm 2. In practice, the set $\mathcal{Z}$ is often simple, such as a box or polyhedron set, allowing an analytical form of the w-update (13c) to be obtained. In such a case, Algorithm 2 completely eliminates the need for an optimization solver, which significantly accelerates the computation.

V. Theoretical Analysis

In this section, for analyzing the convergence of the (s)MEL-ADMM algorithms, we consider the MEL model $\hat{M}_{\frac{1}{\rho}} f$ as an ICNN, ICNN_θ, parameterized by θ as in (9).

To prove Lipschitz continuity of $\nabla \mathrm{ICNN}_\theta(x)$, we consider $\mathrm{ICNN}_\theta(x)$ in the following form

$$\begin{aligned} y_0 &= W_0 x + b_0, \\ v_0 &= \phi_0(y_0) \quad \text{(element-wise)}, \\ y_j &= W_j v_{j-1} + V_j x + b_j, \\ v_j &= \phi_j(y_j) \quad \text{(element-wise)}, \ j = 1, \dots, L, \\ h(x) &= \mathrm{ICNN}_\theta(x) = \phi_L(y_L), \end{aligned}$$

Lemma 2: The ICNN (9) with finite weights has Lipschitz-continuous gradient if all activation functions ϕ_j are continuous twice-differentiable and their derivatives $|\phi_j'|$ and $|\phi_j''|$ are bounded (e.g., softplus, sigmoid, tanh).

Proof: See Appendix A. ∎

We now demonstrate that the MEL-ADMM algorithm always converges. Before stating the main theorem on convergence, we establish the following useful preliminary result.

Lemma 3: Let g be a convex differentiable function, whose gradient is Lipschitz continuous with constant $L = \frac{1}{\lambda}$. There exists a unique $f \in \Gamma_0(\mathbb{R}^n)$ such that $M_\lambda f = g$.

Proof: See Appendix B. ∎

To prove the convergence of Algorithm 1, let us consider its scaled form as follows, with $\bar{\alpha}^i = \frac{1}{\rho}\alpha^i$,

$$\hat{z}^{i+1} = w^i + \bar{\alpha}^i - \rho^{-1}\nabla \mathrm{ICNN}_\theta(w^i + \bar{\alpha}^i), \tag{15a}$$
$$\tilde{z}^{i+1} = \gamma \hat{z}^{i+1} + (1-\gamma) w^i, \tag{15b}$$
$$w^{i+1} = \Pi_{\bar{\mathcal{Z}}}(\tilde{z}^{i+1} - \bar{\alpha}^i), \tag{15c}$$
$$\bar{\alpha}^{i+1} = \bar{\alpha}^i + w^{i+1} - \tilde{z}^{i+1}. \tag{15d}$$

Define $g(x) = \mathrm{ICNN}_\theta(x)$. By Lemma 2, $\nabla g(x)$ is Lipschitz continuous. Let L be a Lipschitz constant of ∇g and suppose that $L \le \rho$. Lemma 3 guarantees a unique function $\hat{f} \in \Gamma_0(\mathbb{R}^n)$ such that $M_{\frac{1}{\rho}} \hat{f} = g$ because ∇g is also ρ-Lipschitz continuous. We then define the following optimization problem

$$\text{minimize } \hat{f}(z) \ \text{ s.t. (1b), (1c).} \tag{16}$$

The feasibility of the constraint set has already been assumed in Section III-A. The following theorem establishes the convergence of MEL-ADMM to a solution of (16).

Theorem 1: If $L \le \rho$, the MEL-ADMM Algorithm 1 converges to an optimal solution of (16).

Proof: Since $M_{\frac{1}{\rho}} \hat{f} = g$, it follows from (7) that (15a) is equivalent to

$$\hat{z}^{i+1} = \mathbf{prox}_{\frac{1}{\rho}\hat{f}}(q^i) = \arg\min_z \hat{f}(z) + \frac{\rho}{2}\|w^i - z + \bar{\alpha}^i\|_2^2.$$

Therefore, the iteration of MEL-ADMM is equivalent to the scaled form of ADMM (4a)-(4d) for solving (16). Since $\bar{\mathcal{Z}}$ is a non-empty set and $\gamma \in (0, 2)$, the scaled ADMM convergences to an optimal solution of (16). ∎

Using a similar argument as for MEL-ADMM, we can establish the convergence of sMEL-ADMM as follows.

Proposition 2: If $L \le \rho$, the sMEL-ADMM Algorithm 2 converges to an optimal solution of (16).

Although (s)MEL-ADMM may not recover an exact solution of (1), its solution is feasible for the constraints of (1). In addition, by informing the properties of the MEL model, as detailed in Section IV-A and summarized in Table I, it can achieve high accuracy of approximating the Moreau envelope, hence the solution returned by (s)MEL-ADMM can get sufficiently close to that of (1). This will be demonstrated in the numerical experiments in Section VII.

Remark 2: Methods proposed in [39], [40] can compute an upper bound L on the Lipschitz constant of $\nabla \mathrm{ICNN}_\theta$, which can be used to verify the condition $L \le \rho$ in the above results.

VI. LEAF for Model Predictive Control

The (s)MEL-ADMM algorithms in the LEAF framework, presented in Section IV, can be applied to accelerate model predictive control (MPC) for linear time-invariant (LTI) systems. Consider the following discrete-time LTI system

$$x_{t+1} = Ax_t + Bu_t, \tag{17}$$

where $x_t \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ is the state, $u_t \in \mathcal{U} \subseteq \mathbb{R}^{n_u}$ is the control input, real matrices A and B are of appropriate dimensions, and t is the time step. MPC [41] computes a feedback control input by solving the constrained optimization problem

$$\text{minimize } J(\mathbf{x}, \mathbf{u}) = c_N(x_{N|t}) + \sum_{k=0}^{N-1} c_k(x_{k|t}, u_{k|t}) \tag{18a}$$
$$\text{s.t. } x_{k+1|t} = Ax_{k|t} + Bu_{k|t}, \tag{18b}$$
$$x_{0|t} = x_t, \tag{18c}$$
$$(x_{k|t}, u_{k|t}) \in \mathcal{X} \times \mathcal{U}, \ k = 0, \dots, N-1, \tag{18d}$$

where $N \in \mathbb{N}_{>0}$ is the prediction horizon, $x_{k|t}$ and $u_{k|t}$ are the predicted state and input at step k in the horizon, $c_k(x_{k|t}, u_{k|t})$ is the stage cost function, $c_N(x_{N|t})$ is the terminal cost function, and $x_{0|t} = x_t$ is the current state. The constraint sets $\mathcal{X}$ and $\mathcal{U}$ are convex. In addition, the cost functions c_k and c_N are convex, proper, and closed. The MPC problem is parameterized by x_t.

Let us define $z_k = [x_{k|t}^\top, u_{k|t}^\top]^\top$, for $k = 0, \dots, N-1$, and $z_N = [x_{N|t}^\top, 0^\top]^\top$. Then, the problem (18) is reformulated as

$$\underset{z_0, \dots, z_N}{\text{minimize}} \sum_{k=0}^{N} (f_k(z_k) + I_{\mathcal{Z}_k}(z_k)) \tag{19a}$$
$$\text{s.t. } M_1 z_0 = x_t, \tag{19b}$$
$$M_1 z_{k+1} - M_2 z_k = 0, \ k = 0, \dots, N-1, \tag{19c}$$

where $f_N(z_N) = c_N(x_{N|t})$, $\mathcal{Z}_N = \mathcal{X} \times \{0\}$, $f_k(z_k) = c_k(x_{k|t}, u_{k|t})$ and $\mathcal{Z}_k = \mathcal{X} \times \mathcal{U}$ for $k = 0, \dots, N-1$, $M_1 = [I_{n_x}, 0_{n_x \times n_u}]$, and $M_2 = [A, B]$.

By defining auxiliary variables $w_k \in \mathbb{R}^{n_x + n_u}$ that are copies of z_k, the problem (19) can be rewritten in consensus form as

$$\text{minimize } J(z, w) = \sum_{k=0}^{N} f_k(z_k) + I_{\mathcal{Z}}(w) \tag{20a}$$
$$\text{s.t. } w - z = 0, \tag{20b}$$

where $z = [z_k]_{k=0,1,\dots,N}$, $w = [w_k]_{k=0,1,\dots,N}$, and $\mathcal{Z} = \{w \in \mathcal{Z}_0 \times \cdots \times \mathcal{Z}_N \,|\, M_1 w_0 = x_t, \ M_1 w_{k+1} - M_2 w_k = 0 \text{ for } k = 0, \dots, N-1\}$.

Applying MEL-ADMM in Section IV-C to (20) yields

$$\hat{z}_k^{i+1} = q_k^i - \rho^{-1}\nabla \hat{M}_{\frac{1}{\rho}} f_k(q_k^i), \ k = 0, 1, \cdots, N, \tag{21a}$$

$$\tilde{z}^{i+1} = \gamma\hat{z}^{i+1} + (1-\gamma)w^i, \tag{21b}$$
$$w^{i+1} = \Pi_{\mathcal{Z}}\left(\hat{z}^{i+1} - \rho^{-1}\alpha^i\right), \tag{21c}$$
$$\alpha^{i+1} = \alpha^i + \rho(w^{i+1} - \tilde{z}^{i+1}), \tag{21d}$$

where $q_k^i = w_k^i + \rho^{-1}\alpha_k^i$ and $\hat{M}_{\frac{1}{\rho}} f_k$ is the MEL model of f_k. In (21), the $\hat{z}$-update, $\tilde{z}$-update, and α-update can be carried out in a parallel manner by k, and the w-update (21c) is a Euclidean projection onto the convex set $\mathcal{Z}$. If the stage cost functions c_k are identical, hence the functions f_k and their Moreau envelopes are also identical, for all $k = 0, \ldots, N-1$, then (21a) is simplified as it requires only two MEL models $\hat{M}_{\frac{1}{\rho}} f_k$ and $\hat{M}_{\frac{1}{\rho}} f_N$.

The MPC problem can also be solved by adopting sMEL-ADMM in Section IV-D. Specifically, define two constraint sets $\mathcal{C}_{\text{bound}} = (\mathcal{X} \times \mathcal{U})^N \times (\mathcal{X} \times \{0\})$ and $\mathcal{C}_{\text{dynamic}} = \{z \in \mathbb{R}^{(n_x+n_u)(N+1)} \mid \mathbf{M}z = d_t\}$, where $\mathbf{M}$ is the $(N+1)\times(N+1)$-block matrix

$$\mathbf{M} = \begin{bmatrix} M_1 & & & \\ -M_2 & M_1 & & \\ & \ddots & \ddots & \\ & & -M_2 & M_1 \end{bmatrix}, \quad d_t = [x_t^\top, 0^\top]^\top.$$

Then (19) can be converted into (12) with $\mathcal{Z} = \mathcal{C}_{\text{bound}}$ and $\mathcal{A} = \mathcal{C}_{\text{dynamic}}$.

LEAF enables computationally efficient solving of the MPC problem (18), especially when the cost functions are convex but non-smooth, as demonstrated in Section VII.

VII. Numerical Results

This section validates the proposed framework through three numerical examples. The first example studies energy management in a microgrid with renewable generation and a battery energy storage system. The second and third examples consider entropy maximization and minimum volume enclosing ellipsoid problems, respectively.

A. Experimental setups

The proposed (s)MEL-ADMM methods are designed to obtain high-quality feasible solutions with low computational cost, rather than to drive the objective value arbitrarily close to the exact optimum. Their accuracy is therefore evaluated through the achieved optimality gap. Feasibility is preserved because the variables are projected onto the admissible search space, ensuring constraint satisfaction. Consequently, the comparisons below focus on accuracy and computational time.

1) Termination condition: Termination criteria differ across optimization methods, particularly between interior-point methods and splitting-operator methods. For example, IPOPT [42] and MadNLP [43] use termination tolerances based on KKT conditions, whereas OSQP [38] and other ADMM implementations use criteria based on primal and dual residuals. Therefore, for each example, we choose termination criteria that support a fair comparison across methods. Specifically, we use the *relative optimality gap*:

$$g_{opt} = \left|\frac{J - J^\star}{J^\star}\right| \times 100\%, \tag{22}$$

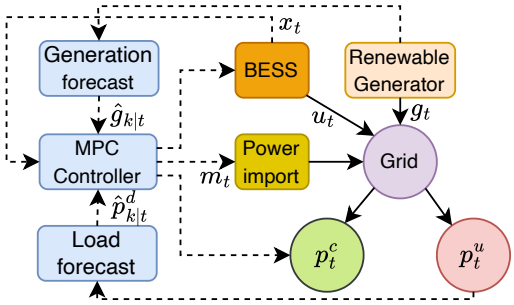


Fig. 2: Example 1 of energy management in a microgrid.

where J is the objective value of the best feasible solution found by a solver and $J^\star$ is the optimal objective value, obtained using high-accuracy interior-point method (IPM) configurations. We then tune the termination tolerances so that all optimizers return feasible solutions within the same prescribed optimality gap.

2) Implementation and baselines: We compare the computational time of (s)MEL-ADMM against several state-of-the-art solvers, including: IPOPT [42], MadNLP [43], Mosek [44], and Clarabel [45]. In addition, we also compare our methods with ADMM [30], implemented as in our methods but without using the MEL model. Ground-truth optimal values for all experiments are obtained using high-accuracy settings of IPOPT and Mosek. All implementations are written in Julia and run on an Apple M4 Pro chip with 24 GB of RAM.

B. Example 1: Energy management in a microgrid

This example considers the microgrid model in Fig. 2 [46], [47], which includes a renewable generator, controllable loads p_t^c, uncontrollable loads p_t^u, a battery energy storage system (BESS), and a utility-grid connection. It is equipped with photovoltaic (PV) generation, allowing it to operate in standalone mode. The control objective is to schedule the BESS to minimize the cost of electricity purchased from the grid while maintaining user comfort for the controllable loads. IPOPT and MadNLP are used as baselines for this example.

Electricity cost: At a prediction step k, the electricity cost includes the energy cost $J_{e,k}$ and the peak demand charge $J_{p,k}$. The energy cost is formulated as

$$J_{e,k} = r_{ec}\Delta_T\left(m_{k|t} + \frac{1-\eta}{2\sqrt{\eta}}|u_{k|t}|\right), \tag{23}$$

where r_{ec}, Δ_T, and η are respectively the energy charge rate, time-step length, and round-trip efficiency factor of the BESS. In addition, $m_{k|t}$ and $u_{k|t}$ denote the predicted imported power and the predicted charging/discharging power of the BESS at k steps ahead of the current time t. The BESS discharges when $u_{k|t} > 0$ and charges when $u_{k|t} < 0$, according to

$$x_{k+1|t} = x_{k|t} - \frac{u_{k|t}}{b_{ess}}\Delta_T, \tag{24}$$

where $b_{ess} > 0$ is the BESS energy capacity (kWh) and $x_{k|t}$ is the predicted state of charge. Let r_{op} be the on-peak demand charge rate. The peak demand charge $J_{p,k}$ is given by

$$J_{p,k} = r_{op}\max\left(m_{k|t}, 0\right). \tag{25}$$

Discomfort cost: At k steps ahead of the current time t, the discomfort cost associated with the controllable load $p^c_{k|t}$ is given by $J_{d,k} = r_{df} f(p^c_{k|t})$, where

$$f(p) = \begin{cases} +\infty, & p \le 0, \\ \frac{a}{p} - 1, & 0 < p < a, \\ 0, & \text{otherwise} \end{cases}$$

measures the customer discomfort level and r_{df} is the discomfort cost weight. Here, a is the preferred power consumption of the controllable load. If $p^c_{k|t}$ is lower than a, the discomfort level is high, and the discomfort decreases as the supplied power approaches the preferred level.

For an N-step horizon, the objective is to minimize the total electricity and discomfort cost,

$$J = \sum_{k=1}^{N} (J_{e,k} + J_{p,k} + J_{d,k}). \tag{26}$$

Let $\hat{g}_{k|t}$ and $\hat{p}^u_{k|t}$ denote the forecasted renewable generation and forecasted uncontrollable demand at prediction step k. The power balance is then given by

$$p^c_{k|t} + \hat{p}^u_{k|t} = m_{k|t} + u_{k|t} + \hat{g}_{k|t}. \tag{27}$$

The renewable generation and demand are forecasted over the prediction horizon at each time step t. The receding horizon planning problem is thus formulated as:

$$\text{minimize } J(\{p^c_{k|t}, u_{k|t}, m_{k|t}\}_{k=1,\dots,N}) \tag{28a}$$
$$\text{s.t. } (24), (27), \tag{28b}$$
$$u_{\max} \ge u_{k|t} \ge u_{\min}, \tag{28c}$$
$$x_{\max} \ge x_{k|t} \ge x_{\min},\ x_{N|t} \ge \underline{x}_N. \tag{28d}$$

Because our methods can handle the nonsmooth cost (28a), we use (28) to collect data for training the MEL model described in Section IV-C. To apply IPOPT and MadNLP, however, we transform (28) into a smooth problem by introducing auxiliary variables $s_{u,k}$, $s_{m,k}$ and $s_{d,k}$ then reformulating (28) as

$$\text{minimize } \sum_{k=1}^{N} r_{ec}\Delta_T \left(m_{k|t} + \frac{1-\eta}{2\sqrt{\eta}} s_{u,k} \right) + r_{op} s_{m,k} + r_{df} s_{d,k} \tag{29a}$$
$$\text{s.t. } (24), (27), (28c) \text{ and } (28d), \tag{29b}$$
$$s_{u,k} \ge u_{k|t},\ s_{u,k} \ge -u_{k|t}, \tag{29c}$$
$$s_{m,k} \ge m_{k|t},\ s_{m,k} \ge 0, \tag{29d}$$
$$s_{d,k} \ge \frac{a}{p^c_{k|t}} - 1,\ s_{d,k} \ge 0. \tag{29e}$$

By the definition of the discomfort function, the controllable load $p^c_{k|t}$ cannot be negative, otherwise the objective will become $+\infty$. Therefore, (28) is equivalent to (29), which has a linear objective function by moving the nonlinear terms introduced by the discomfort model to the constraints.

For this setup, we use the following parameters (partly taken from [47]): $\Delta_T = 0.25\,\text{h}$, $N = 96$ (corresponding to 24 hours), $r_{ec} = \$0.1/\text{kWh}$, $r_{op} = \$19.19/\text{kW}$, $r_{df} = \$10$, initial SOC $x_0 = 0.5$ (50%), $u_{\max} = 700\,\text{kW}$, $u_{\min} = -700\,\text{kW}$, $b_{ess} = 500\,\text{kWh}$, $\eta = 0.8$, $x_{\min} = 0.2$ (20%), $x_{\max} = 0.8$ (80%), $\underline{x}_N = 0.5$ (50%), and $a = 50\,\text{kW}$. Perfect forecasts are assumed for power demand and PV generation. Using IPOPT with $\epsilon_{tol} = 10^{-8}$, the optimal objective values for $N = 96$ (one day) and $N = 192$ (two days) are $J^\star_{N=96} = 36479.1$ and $J^\star_{N=192} = 73117.5$, respectively.

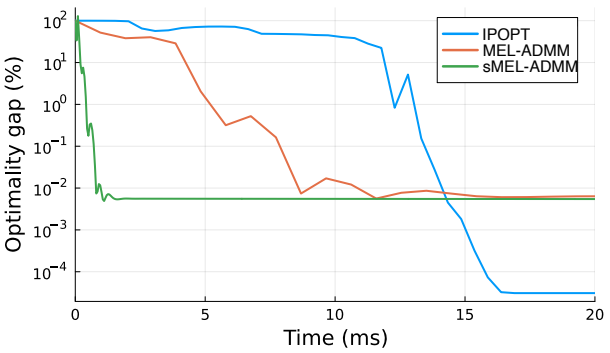


Fig. 3: Optimality gap over solving time for Example 1.

Table II shows that both methods of our framework consistently outperform the classical IPM baselines. For the one-day horizon case ($N = 96$), MEL-ADMM attains approximately a $2\times$ speedup over the IPM baselines within optimality gap $g_{opt} = 1\%$ and maintains the advantage as the tolerance tightens to $g_{opt} = 0.01\%$; sMEL-ADMM further reduces the solving time by more than ten times while preserving feasibility. Doubling the horizon to $N = 192$ amplifies these trends: the runtimes of IPOPT, MadNLP, and conventional ADMM roughly double, whereas MEL-ADMM scales sublinearly and sMEL-ADMM remains near one millisecond on average, underscoring the benefit of Moreau envelope learning in capturing the structure induced by the battery and comfort constraints. The small standard deviations confirm that the proposed algorithms deliver stable execution times even as the optimality requirements change.

Figure 3 compares the solvers in terms of accuracy versus time and shows that MEL-ADMM and sMEL-ADMM reach the target gap much more rapidly than IPOPT. For high and low optimality gaps ($g_{opt} \le 1\%$ and $g_{opt} \le 0.01\%$), sMEL-ADMM reaches the desired accuracy almost immediately, while IPOPT requirs a longer transient before the optimality gap becomes sufficiently small. This behavior demonstrates the ability of sMEL-ADMM to obtain feasible suboptimal solutions rapidly within acceptable accuracy requirements. While MEL-ADMM and sMEL-ADMM plateau once the gap target drops below $10^{-3}\%$, IPOPT continues converging and reliably attains gaps under $10^{-4}\%$.

The trajectories in Fig. 4 further validate that the solution obtained by sMEL-ADMM satisfies the BESS operating requirements. The algorithm keeps the BESS SOC within $[20\%, 80\%]$ and enforces power balance by co-adjusting $m_{k|t}$ and $p^c_{k|t}$. During periods of high PV injections (daytime hours 8–14), the algorithm reduces imports and charges the battery aggressively, while the discomfort cost stays bounded. In the evening peak, the algorithm reverses power flow, discharging the battery to shave the grid purchases without violating the terminal constraint on $x_{N|t}$.

TABLE II: Computation time (in ms) benchmark for solving the optimization problem in VII-B.

	IPOPT	MadNLP	ADMM	MEL-ADMM	sMEL-ADMM
N = 96, $g_{opt} = 1\%$	11.5 ± 0.4	15.3 ± 0.2	80.1 ± 1.4	6.3 ± 0.1	**0.5 ± 0.1**
N = 96, $g_{opt} = 0.01\%$	13.3 ± 0.3	18.5 ± 0.2	111.4 ± 1.8	8.9 ± 0.1	**0.9 ± 0.1**
N = 192, $g_{opt} = 1\%$	24.3 ± 0.1	35.3 ± 0.3	159.5 ± 2.1	11.7 ± 0.1	**0.7 ± 0.1**
N = 192, $g_{opt} = 0.01\%$	26.1 ± 0.2	37.6 ± 0.3	215.8 ± 2.3	17.6 ± 0.2	**1.3 ± 0.2**

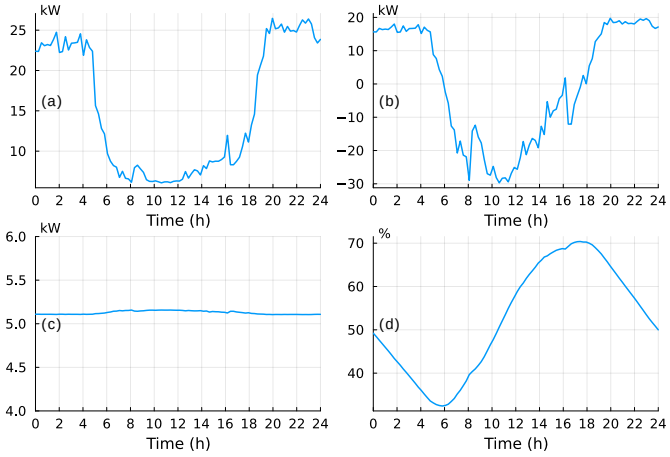
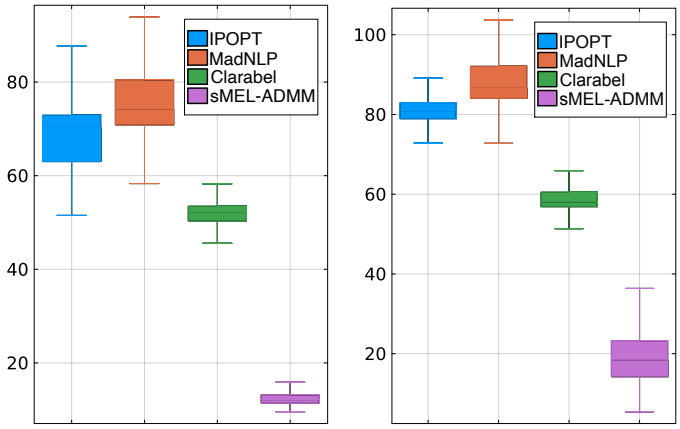


Fig. 4: Example 1: Prediction over 24-hour (96-step) horizon of (a) imported power $m_{k|t}$, (b) discharging power $u_{k|t}$, (c) controllable load $p^c_{k|t}$, and (d) SOC of BESS.



(a) $g_{opt} \leq 0.1\%$ (b) $g_{opt} \leq 0.01\%$

Fig. 5: Solving time (ms) for the entropy maximization problem ($n = 1000, m = 100$) using IPOPT, MadNLP, Clarabel, and sMEL-ADMM over 1000 randomized parameter instances and two optimality gaps.

C. Example 2: Entropy maximization

This example considers the constrained entropy maximization problem in [48, Eq. (5.13)], expressed as

$$\underset{x>0}{\text{maximize}} \; -\sum_{i=1}^{n} x_i \log x_i \tag{30a}$$

$$\text{s.t. } \mathbf{1}^\top x = 1, \tag{30b}$$

$$Ax \leq b, \tag{30c}$$

where $x = [x_1, x_2, \ldots, x_n] \in \mathbb{R}^n$ is the decision variable representing a probability distribution, and $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ define the linear inequality constraints. Here, A and b are treated as problem parameters. To apply sMEL-ADMM (Algorithm 2) to this problem, we reformulate (30c) as $Ax = s$ and $s \leq b$ with auxiliary variable $s \in \mathbb{R}^m$.

This example evaluates the validity, accuracy, and computational performance of sMEL-ADMM under randomized problem parameters. Optimal objective values are obtained from high-accuracy IPOPT solves. The proposed sMEL-ADMM is compared with IPOPT and two modern conic or nonlinear solvers, MadNLP and Clarabel, which constitute the baseline solvers. All elements of A are sampled uniformly from $[0, 1]$, and b is randomly selected so that (30c) is feasible. We solve (30) with different dimensions $n = 10^2, 10^3, 10^4$ and $m = 1, 10, 10^2$, different accuracy targets $g_{opt} \leq 1\%, 0.1\%, 0.01\%$, and 10,000 different (A, b) samples. The other sMEL-ADMM parameters are $\gamma = 1.6$ and $\rho = 1$ for all experiments.

Fig. 5 shows the distribution of solving times of the solvers with $n = 1000$ and $m = 100$ over 1000 randomly generated instances under two optimality tolerances. In both accuracy regimes, sMEL-ADMM exhibits a substantially smaller median solving time than IPOPT, MadNLP, and Clarabel, indicating faster and more stable performance. While IPOPT and MadNLP show comparable median runtimes with relatively wide variability, Clarabel is consistently faster than these two but remains significantly slower than sMEL-ADMM. As the optimality requirement becomes stricter ($g_{\text{opt}} \leq 0.01\%$), the computational cost of all solvers increases; however, sMEL-ADMM maintains highly competitive solving times compared to the baseline methods, highlighting the scalability and robustness of sMEL-ADMM for high-accuracy solutions.

Table III reports the solving times of IPOPT and sMEL-ADMM over 1000 randomized parameter pairs (A, b) for three optimality targets. The results show that sMEL-ADMM yields faster solving times than IPOPT at low ($g_{opt} \leq 1\%$), medium ($g_{opt} \leq 0.1\%$), and high ($g_{opt} \leq 0.01\%$) accuracy levels. For instance, for $(n = 10^4, m = 100)$, sMEL-ADMM is over ten times faster than IPOPT across all three accuracy targets.

D. Example 3: Minimum volume enclosing ellipsoid (MVEE)

This example considers the MVEE problem [49]

$$\underset{P}{\text{minimize}} \; \det(P^{-1}) \tag{31a}$$

$$\text{s.t. } (x_i - c)^\top P (x_i - c) \leq 1, \; i = 1, \ldots, m, \tag{31b}$$

$$P \succ 0, \tag{31c}$$

where $x_i \in \mathbb{R}^n$ is the i-th point, $P = P^\top \in \mathbb{R}^{n \times n}$ is a positive definite matrix that defines the shape of the ellipsoid, and $c \in \mathbb{R}^n$ is the center of the ellipsoid. By introducing auxiliary variables $s_i \in \mathbb{R}$, (31) can be rewritten as

$$\underset{P}{\text{minimize}} \; \det(P^{-1}) \tag{32a}$$

TABLE III: Average solving time (ms) of IPOPT and sMEL-ADMM for Example 2 (entropy maximization) over 1000 randomized parameter instances.

g_{opt}	n	m	IPOPT	sMEL-ADMM
$\leq 1\%$	10^2	1	0.88	**0.12**
		10	1.12	**0.14**
	10^3	10	6.69	**0.45**
		100	27.47	**5.71**
	10^4	10	54.23	**6.59**
		100	595.01	**55.31**
$\leq 0.1\%$	10^2	1	0.93	**0.13**
		10	1.39	**0.22**
	10^3	10	8.36	**0.49**
		100	62.82	**10.46**
	10^4	10	54.27	**6.59**
		100	1395.38	**88.63**
$\leq 0.01\%$	10^2	1	1.09	**0.19**
		10	1.75	**0.58**
	10^3	10	9.33	**0.51**
		100	73.12	**14.34**
	10^4	10	57.1	**15.9**
		100	1597.93	**147.64**

$$\text{s.t. } H_i^\top \operatorname{vec}(P) = s_i,\ i = 1, \ldots, m, \tag{32b}$$
$$P \succ 0, \quad s_i \leq 1, \tag{32c}$$

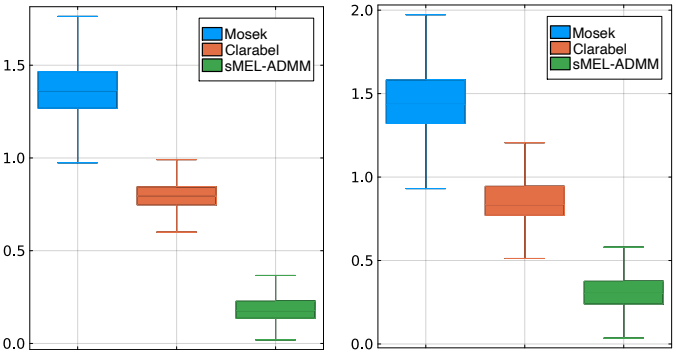
where $\operatorname{vec}(P)$ concatenates the diagonal and upper-triangular elements of the symmetric matrix P into a vector. In (32), $H_i = [v_{i,1}^2, 2v_{i,1}v_{i,2}, v_{i,2}^2, 2v_{i,1}v_{i,3}, 2v_{i,2}v_{i,3}, v_{i,3}^2, \ldots\ v_{i,n}^2]^\top \in \mathbb{R}^{\frac{n(n+1)}{2}}$ with $v_i = [v_{i,j}]_{1\leq j\leq n} = x_i - c$. With this setup, sMEL-ADMM can be applied to solve (32). To simplify the experiment without loss of generalization, we set $n = 3$, $c = 0$, and sample each x_i uniformly from $[-1, 1]$.

Two leading semidefinite programming solvers, Mosek and Clarabel, are selected as baselines for this example. Table IV reports the computation time of the proposed sMEL-ADMM method and the baseline solvers over 1000 randomly generated problem instances. Across all scenarios, sMEL-ADMM consistently outperforms the baseline solvers. Specifically, at $m = 55$ with $g_{opt} = 1\%$, sMEL-ADMM is approximately 7.5 times faster than Mosek and 4 times faster than Clarabel. Even when the optimality gap is tightened to 0.1%, sMEL-ADMM maintains its advantage, solving (32) about 5 times faster than Mosek and nearly 3 times faster than Clarabel. As the number of constraints increases, all solvers exhibit higher solving times; however, sMEL-ADMM preserves its performance advantage. These results emphasize the efficiency and scalability of our learning-enabled framework.

Fig. 6 provides a statistical breakdown of the solving time with $m = 55$ under optimality gaps of 1% and 0.1%. While Table IV summarizes average performance, these box plots reveal the consistency of sMEL-ADMM. The interquartile range of sMEL-ADMM is significantly narrower and lower than those of Mosek and Clarabel. Even in the more stringent optimality gap scenario, sMEL-ADMM maintains a much tighter distribution compared to the baseline solvers, whose solving times show wider fluctuations. These results demonstrate that the proposed approach achieves stable, high-speed computation under randomized parameter conditions.

TABLE IV: Average solving time (ms) of Mosek, Clarabel, and sMEL-ADMM for Example 3 (MVEE) over 1000 randomized parameter instances.

	Mosek	Clarabel	sMEL-ADMM
m = 55, $g_{opt} = 1\%$	1.36	0.79	**0.18**
m = 55, $g_{opt} = 0.1\%$	1.44	0.84	**0.30**
m = 75, $g_{opt} = 1\%$	1.66	0.89	**0.23**
m = 75, $g_{opt} = 0.1\%$	1.67	0.92	**0.38**



(a) Optimality gap $\leq 1\%$ (b) Optimality gap $\leq 0.1\%$

Fig. 6: Solving time for the MVEE problem with $m = 55$ over 1000 randomized parameter instances.

VIII. Conclusion

This paper presented LEAF, a learning-enabled ADMM framework for accelerated convex optimization through learned approximations of the Moreau envelope (ME). By integrating an ICNN-based ME model into ADMM iterations, the proposed framework preserves key structural properties of convex optimization, including convexity, smoothness, feasibility, and convergence guarantees. Unlike existing learning-to-optimize approaches that directly approximate high-dimensional operators, LEAF learns a scalar-valued ME function, substantially reducing model complexity while improving scalability and data efficiency.

Based on LEAF, we developed MEL-ADMM and sMEL-ADMM with established theoretical guarantees. Numerical experiments on large-scale optimization problems demonstrated that the proposed methods achieve significant computational speedups over state-of-the-art solvers while maintaining low optimality gaps and strict constraint satisfaction. The results further show that exploiting optimization structure within the learning architecture can substantially improve both efficiency and reliability in learning-enabled optimization.

Future work will investigate extensions to broader problem classes, including mixed-integer and distributed convex optimization. We also plan to explore applications in robotics, autonomous systems, and real-time control, where fast and reliable optimization is critical.

References

[1] T. Chen, X. Chen, W. Chen, H. Heaton, J. Liu, Z. Wang, and W. Yin, "Learning to optimize: A primer and a benchmark," *Journal of Machine Learning Research*, vol. 23, no. 189, pp. 1–59, 2022.

[2] J. Sacks and B. Boots, "Learning to optimize in model predictive control," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 10 549–10 556.

[3] H. Wang, R. Feng, Z.-F. Han, and C.-S. Leung, "Admm-based algorithm for training fault tolerant rbf networks and selecting centers," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3870–3878, 2017.

[4] T. B. Nguyen, T. Nguyen, T. Nghiem, L. Nguyen, J. Baca, P. Rangel, and H.-K. Song, "Collision-free minimum-time trajectory planning for multiple vehicles based on admm," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 13 785–13 790.

[5] A. Duarte, B. Nguyen, T. X. Nghiem, and S. Wei, "Communication-efficient ADMM using Gaussian process regression and fully adaptive uniform quantization," *Franklin Open*, vol. 15, p. 100587, 2026.

[6] W. An, Y. Liu, F. Shang, H. Liu, and L. Jiao, "Des-inspired accelerated unfolded linearized admm networks for inverse problems," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 3, pp. 5319–5333, 2025.

[7] M. Hasanzadeh and A. Kargarian, "ADMM Enhancement Techniques for Distributed Optimal Power Flow," *IEEE Transactions on Power Systems*, vol. 41, no. 2, pp. 1321–1333, Mar. 2026.

[8] P. Xu, Y. Wang, X. Chen, and Z. Tian, "Qc-odkla: Quantized and communication- censored online decentralized kernel learning via linearized admm," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 12, pp. 17 987–17 999, 2024.

[9] T. Zhang, S. Ye, X. Feng, X. Ma, K. Zhang, Z. Li, J. Tang, S. Liu, X. Lin, Y. Liu, M. Fardad, and Y. Wang, "StructADMM: Achieving ultrahigh efficiency in structured pruning for DNNs," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 5, pp. 2259–2273, 2022.

[10] B. Nguyen, T. X. Nghiem, L. Nguyen, H. M. La, and T. Nguyen, "Connectivity-preserving distributed informative path planning for mobile robot networks," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2949–2956, 2024.

[11] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: a methodological tour d'horizon," *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.

[12] S. Koziel and L. Leifsson, *Surrogate-based modeling and optimization*, 1st ed. Springer New York, NY, 2013.

[13] A. S. Zamzam and K. Baker, "Learning optimal solutions for extremely fast ac optimal power flow," in *2020 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids*, Nov. 2020, pp. 1–6.

[14] S. Adhau, V. V. Naik, and S. Skogestad, "Constrained neural networks for approximate nonlinear model predictive control," in *2021 60th IEEE Conference on Decision and Control (CDC)*, 2021, pp. 295–300.

[15] V. Peiris, N. Sukhorukova, and J. Ugon, "KKT-based optimality conditions for neural network approximation," *arXiv preprint 2506.17305*, 2025.

[16] P. L. Donti, D. Rolnick, and J. Z. Kolter, "DC3: A learning method for optimization with hard constraints," in *International Conference on Learning Representations*, 2021.

[17] H. T. Nguyen and P. L. Donti, "FSNet: Feasibility-seeking neural network for constrained optimization with guarantees," in *Annual Conference on Neural Information Processing Systems*, 2026.

[18] K. Baker, "Learning warm-start points for AC Optimal Power Flow," in *2019 IEEE 29th International Workshop on Machine Learning for Signal Processing (MLSP)*, Oct. 2019, pp. 1–6.

[19] R. Sambharya, G. Hall, B. Amos, and B. Stellato, "Learning to warm-start fixed-point optimization algorithms," *Journal of Machine Learning Research*, vol. 25, no. 166, pp. 1–46, 2024.

[20] S. Misra, L. Roald, and Y. Ng, "Learning for constrained optimization: Identifying optimal active constraint sets," *INFORMS Journal on Computing*, vol. 34, no. 1, pp. 463–480, 2022.

[21] J. R. Chang, C.-L. Li, B. Póczos, B. Vijaya Kumar, and A. C. Sankaranarayanan, "One network to solve them all — solving linear inverse problems using deep projection models," in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 5889–5898.

[22] T. Meinhardt, M. Moller, C. Hazirbas, and D. Cremers, "Learning proximal operators: Using denoising networks for regularizing inverse imaging problems," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 1781–1790.

[23] J. Sun, H. Li, Z. Xu *et al.*, "Deep ADMM-Net for compressive sensing MRI," *Advances in neural information processing systems*, vol. 29, 2016.

[24] Y. Yang, J. Sun, H. Li, and Z. Xu, "ADMM-CSNet: A deep learning approach for image compressive sensing," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 3, pp. 521–538, 2020.

[25] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, B. Shillingford, and N. De Freitas, "Learning to learn by gradient descent by gradient descent," *Advances in neural information processing systems*, vol. 29, 2016.

[26] Y. Chen, M. W. Hoffman, S. G. Colmenarejo, M. Denil, T. P. Lillicrap, M. Botvinick, and N. Freitas, "Learning to learn without gradient descent by gradient descent," in *International Conference on Machine Learning*. PMLR, 2017, pp. 748–756.

[27] S. Park and P. Van Hentenryck, "Self-supervised primal-dual learning for constrained optimization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 4052–4060.

[28] Y. Min and N. Azizan, "Hardnet: Hard-constrained neural networks with universal approximation guarantees," *arXiv preprint 2410.10807*, 2024.

[29] R. T. Rockafellar, *Convex analysis*. Princeton University Press, 1997.

[30] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, Jul. 2011.

[31] R. Nishihara, L. Lessard, B. Recht, A. Packard, and M. Jordan, "A general analysis of the convergence of ADMM," in *Proceedings of the 32nd International Conference on Machine Learning*, vol. 37. PMLR, 2015, pp. 343–352.

[32] J. Eckstein and M. C. Ferris, "Operator-splitting methods for monotone affine variational inequalities, with a parallel application to optimal control," *INFORMS Journal on Computing*, vol. 10, no. 2, pp. 218–235, 1998.

[33] H. H. Bauschke and P. L. Combettes, *Convex analysis and monotone operator theory in Hilbert spaces*, 2nd ed. Springer Cham, 2020.

[34] D. P. Bertsekas, *Convex Optimization Algorithms*. Athena Scientific, 2015.

[35] T. X. Nghiem, G. Stathopoulos, and C. N. Jones, "Learning proximal operators with gaussian processes," in *2018 56th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, 2018, pp. 935–942.

[36] B. Amos, L. Xu, and J. Z. Kolter, "Input convex neural networks," in *International conference on machine learning*, 2017, pp. 146–155.

[37] C. Planiden and X. Wang, "Strongly Convex Functions, Moreau Envelopes, and the Generic Nature of Convex Functions with Strong Minimizers," *SIAM Journal on Optimization*, vol. 26, no. 2, pp. 1341–1364, Jan. 2016.

[38] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "Osqp: An operator splitting solver for quadratic programs," in *2018 UKACC 12th International Conference on Control (CONTROL)*, 2018, pp. 339–339.

[39] A. Virmaux and K. Scaman, "Lipschitz regularity of deep neural networks: analysis and efficient estimation," *Advances in neural information processing systems*, vol. 31, 2018.

[40] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. Pappas, "Efficient and accurate estimation of lipschitz constants for deep neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[41] D. Q. Mayne, "Model predictive control: Recent developments and future promise," *Automatica*, vol. 50, no. 12, pp. 2967–2986, 2014.

[42] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.

[43] F. Pacaud and S. Shin, "GPU-accelerated dynamic nonlinear optimization with ExaModels and MadNLP," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*. IEEE, 2024, pp. 5963–5968.

[44] E. D. Andersen and K. D. Andersen, "The MOSEK interior point optimizer for linear programming: an implementation of the homogeneous algorithm," in *High performance optimization*. Springer, 2000, pp. 197–232.

[45] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives," *arXiv preprint 2405.12762*, 2024.

[46] M. J. Risbeck and J. B. Rawlings, "Economic model predictive control for time-varying cost and peak demand charge optimization," *IEEE Transactions on Automatic Control*, vol. 65, no. 7, pp. 2957–2968, 2020.

[47] C. Cortes-Aguirre, Y.-A. Chen, A. Ghosh, J. Kleissl, and A. Khurram, "Economic MPC with an Online Reference Trajectory for Battery Scheduling Considering Demand Charge Management," *IEEE Transactions on Smart Grid*, pp. 1–1, 2025.

[48] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.

[49] N. Bowman and M. T. Heath, "Computing minimum-volume enclosing ellipsoids," *Mathematical Programming Computation*, vol. 15, no. 4, pp. 621–650, 2023.

[50] R. T. Rockafellar and R. J. Wets, *Variational analysis*, 1st ed. Springer Berlin, Heidelberg, 1997.

[51] J. Borwein and A. Lewis, *Convex Analysis and Nonlinear Optimization: Theoryand Examples*, 2nd ed. Springer New York, NY, 2005.

APPENDIX A
PROOF OF LEMMA 2

We will show that $\nabla_x^2 h(x)$ is uniformly bounded. We first derive the expressions for $\nabla_x h(x)$ and $\nabla_x^2 h(x)$. Let $D_j = \operatorname{diag}\big(\phi_j'(y_j)\big)$ and $E_j = \operatorname{diag}\big(\phi_j''(y_j)\big)$. The Jacobian $J_j = \frac{\partial v_j}{\partial x}$ is given by the chain rule as follows

$$\begin{aligned} J_0 &= D_0 W_0, \\ J_j &= D_j\big(W_j J_{j-1} + V_j\big),\ j = 1, \dots, L-1, \\ G_L &= \nabla_x y_L = W_L J_{L-1} + V_L, \\ \nabla_x h &= D_L G_L^\top = \phi_L'(y_L) G_L^\top. \end{aligned}$$

Using the chain rule again, the second derivative (Hessian) for the output layer is derived as follows

$$\begin{aligned} \nabla_x^2 h &= \phi_L''(y_L) \nabla_x y_L \nabla_x y_L^\top + \phi_L'(y_L) \nabla_x^2 y_L \\ &= \phi_L''(y_L)\, G_L^\top G_L + \phi_L'(y_L) \sum_{k=1}^{n_{L-1}} (W_L)_k\, \nabla_x^2 (v_{L-1})_k, \end{aligned}$$

where $(\cdot)_k$ denotes the k-th row (element) of a matrix (vector), and n_j is the number of neurons at the j-th layer. For the i-th neuron in layer j, we define $g_{j,i}^\top = \big(W_j J_{j-1} + V_j\big)_i$, then

$$\nabla_x^2 (v_j)_i = \phi_j''\big((y_j)_i\big)\, g_{j,i} g_{j,i}^\top + \phi_j'\big((y_j)_i\big)\, \nabla_x^2 (y_j)_i,$$

with $\nabla_x^2 (y_j)_i = \sum_{k=1}^{n_{j-1}} ((W_j)_i)_k\, \nabla_x^2 (v_{j-1})_k$.

In the input layer $j = 0$, we have $g_{0,i}^\top = (W_0)_i$, $\nabla_x^2 (v_0)_i = \phi_0''\big((z_0)_i\big)\, (W_0)_i^\top (W_0)_i$. Because all weights are finite and ϕ_ℓ' and ϕ_ℓ'' are bounded, each term in the summation remains bounded. For instance, $\|\nabla_x^2 (v_0)_i\| \le H_0$ implies that

$$\|\partial_y^2 (v_1)_i\| \le \|\phi_1''((y_1)_i)\| \|g_{1,i} g_{1,i}^\top\| + \|\phi_1'((y_1)_i)\| H_0 \|W_1\|_\infty.$$

Finally, by applying the mean-value theorem, it follows that

$$\|\nabla_x h(x_1) - \nabla_x h(x_2)\|_2 \le \sup_x \|\nabla_x^2 h(x)\|\, \|x_1 - x_2\|_2,$$

which proves that $\nabla_x h$ is globally Lipschitz-continuous.

APPENDIX B
PROOF OF LEMMA 3

We first visit some useful definitions and results.

Definition 2 (Convex conjugate): The convex conjugate of a function f is defined as $f^\star(y) = \sup_{x \in \mathbb{R}^n} \{x^\top y - f(x)\}$.

Lemma 4 (Fenchel biconjugate [50]): The Fenchel biconjugate of f is the convex conjugate of $f^\star$, i.e., $f^{\star\star}(x) = \sup_{y \in \mathbb{R}^n} \{y^\top x - f^\star(y)\}$. If $f \in \Gamma_0(\mathbb{R}^n)$ then $f^{\star\star} = f$.

Lemma 5 ([50] Proposition 12.60): For a convex differentiable f, ∇f is Lipschitz continuous with constant $\frac{1}{\sigma}$ if and only if $f^\star$ is σ-strongly convex.

Lemma 6 (Infimal convolution [51](Exercise 3.12)): For $f, g \in \Gamma_0(\mathbb{R}^n)$, we define the *infimal convolution* $(f \odot g)(y) = \inf_x \{f(x) + g(y - x)\}$. Then, the following statements hold

(i) $f \odot g \in \Gamma_0(\mathbb{R}^n)$,
(ii) $(f \odot g)^\star = f^\star + g^\star$.

Now, we are ready to prove Lemma 3. We prove *existence* and *uniqueness* separately.

Existence: According to Lemma 5 (convex differentiable functions from $\mathbb{R}^n \mapsto \mathbb{R}$ belong to $\Gamma_0(\mathbb{R}^n)$), $g^\star$ is a λ-strongly convex function. Let us define a function $f = (g^\star - \frac{\lambda}{2}\|.\|_2^2)^\star$. By Definition 1, $g^\star - \frac{\lambda}{2}\|.\|_2^2$ is a convex function. By Lemma 4, $f^\star = (g^\star - \frac{\lambda}{2}\|.\|_2^2)^{\star\star} = g^\star - \frac{\lambda}{2}\|.\|_2^2$. Therefore, $f \in \Gamma_0(\mathbb{R}^n)$. Based on Lemma 6, the ME is a special case of infimal convolution with the function $q_\lambda(x) = \frac{1}{2\lambda}\|x\|_2^2$. Since $q_\lambda^\star(y) = \frac{\lambda}{2}\|y\|_2^2$, we have

$$(M_\lambda f)^\star = f^\star + q_\lambda^\star = f^\star + \frac{\lambda}{2}\|.\|_2^2 = g^\star.$$

Hence, $M_\lambda f = g$.

Uniqueness: Assume there exists another $\hat{f} \in \Gamma_0(\mathbb{R}^n)$ such that $M_\lambda \hat{f} = g$. Using the same conjugate identity,

$$g^\star = (M_\lambda \hat{f})^\star = \hat{f}^\star + \frac{\lambda}{2}\|.\|_2^2.$$

Thus $\hat{f}^\star = g^\star - \frac{\lambda}{2}\|.\|_2^2 = f^\star$. By Lemma 4, $\hat{f} = \hat{f}^{\star\star} = f^{\star\star} = f$.